



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105.
Technical Presentation
CSA0314 - DATA STRUCTURES

Guided by:
Dr. Krishnaraj Mony

Presented By :
T. Arhan(192525301)

Problem & Requirement

- Problem:
- A messaging application temporarily stores recent chat messages for quick access.
- Messages are added as new chats arrive and older messages may need to be removed.
- The application should access the most recent messages quickly.
- Requirements:
- Store recent messages temporarily.
- Add new messages efficiently.
- Remove the oldest messages when storage is full.
- Access recent messages quickly.

Suitable Data Structure: Circular Queue

- A Circular Queue is a linear data structure that follows FIFO (First In, First Out).
- Front → points to the oldest message.
- Rear → points to the newest message.
- When the queue becomes full, the oldest message can be removed as a new message is added.
- The circular arrangement reuses available positions efficiently.

Operations & Efficiency

- Enqueue (add message): $O(1)$ time.
- Dequeue (remove oldest message): $O(1)$ time.
- Peek/access the front message: $O(1)$ time.
- Checking the newest message at the rear: $O(1)$ time.
- Space Complexity: $O(n)$, where n is the maximum number of recent messages stored.
- Therefore, a circular queue provides fast operations with fixed temporary storage.

Why Circular Queue is Appropriate?

Data Structure	Suitability	Reason
Array	Moderately suitable	Fast access, but managing oldest messages needs extra logic.
Linked List	Suitable	Easy insertion/deletion, but extra memory for links.
Stack	Less suitable	LIFO does not match oldest-message removal.
Queue	Highly suitable	FIFO matches recent-message storage and removal.
Circular Queue	Highly suitable	FIFO + efficient reuse of fixed storage.

Advantages of Circular Queue

- Maintains messages in chronological order.
- New messages are added at the rear.
- Oldest messages are removed from the front.
- Insertion and deletion can be performed in $O(1)$ time.
- Fixed-size storage prevents unlimited memory growth.
- Circular reuse avoids wasting array positions.
- Suitable for temporary caches, recent-message lists, and streaming data.

Example

- Suppose the application stores only 5 recent messages.
- $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$
- When $M6$ arrives, $M1$ (the oldest) is removed.
- Queue becomes: $M2 \rightarrow M3 \rightarrow M4 \rightarrow M5 \rightarrow M6$
- When $M7$ arrives, $M2$ is removed.
- Queue becomes: $M3 \rightarrow M4 \rightarrow M5 \rightarrow M6 \rightarrow M7$
- This keeps only the latest messages while using fixed storage.

Conclusion

- A Circular Queue is appropriate for temporarily storing recent chat messages.
- Messages are maintained in FIFO order.
- New messages are inserted at the rear and the oldest messages are removed from the front.
- Enqueue, dequeue, and front/rear operations can be $O(1)$.
- Space complexity is $O(n)$ for n stored messages.
- It provides simple and efficient management of a fixed-size recent-message cache.

THANK YOU